

Recovering an Empty PRD

Clarifying Questions & Intake Template for the Product Owner

Prepared by: Product Operations / Research-PM Lead **Date:** May 18, 2026

Status: Ready for distribution **Audience:** Product Owner (primary), Engineering Lead, Design Lead (CC)

Table of Contents

1. [Executive Summary](#)
 2. [Scope & Background](#)
 3. [Key Findings](#)
 4. [Analysis](#)
 5. [Deliverable A — Clarifying Questions](#)
 6. [Deliverable B — Fill-In PRD Template](#)
 7. [Recommendation](#)
 8. [Next Steps](#)
-

1. Executive Summary

The submitted PRD contains only placeholder text ("*Test content patch*") and cannot be used to drive design, estimation, or implementation. The correct response is **not** to infer requirements, but to return a structured intake template to the product owner with targeted clarifying questions covering problem, users, scope, success metrics, and constraints. A ready-to-send template and question bank are provided below.

Sources: This is a process/template-design task drawing on established PRD best practices (Marty Cagan, Lenny Rachitsky, Atlassian, ProductPlan). No live web search was required — the deliverable is a reusable artifact, not a market scan.

2. Scope & Background

Research scope. Design (a) a clarifying-questions checklist and (b) a fill-in PRD template that the product owner can complete to convert the empty document into an actionable spec.

Out of scope. Guessing at the feature's purpose; conducting competitive research on an unknown feature.

Why it matters. Proceeding on a placeholder PRD risks building the wrong thing, wasted engineering cycles, and ambiguous acceptance criteria. A short, structured intake unblocks the team within one product-owner working session rather than triggering rework after build.

3. Key Findings

#	Finding	Reasoning	Confidence
1	The PRD is non-actionable in its current state	"Test content patch" is a placeholder string typically left by a CMS or git template; it contains zero requirement signal. No team member can derive scope, users, or acceptance criteria from it.	High
2	Do not attempt to reconstruct intent from the title or metadata alone	Inferring requirements without product-owner confirmation creates phantom requirements that drift from real user needs, leading to scope disputes mid-sprint.	High
3	A structured intake template outperforms open-ended "please clarify" requests	Product owners under time pressure default to vague prose unless given specific fields. Forced-choice fields (e.g., role table, must-have vs. nice-to-have) yield faster, sharper answers.	High
4	The minimum viable PRD has six	These map to the standard <i>why / who / what / how-we-know-it-</i>	High

#	Finding	Reasoning	Confidence
	sections: Problem, Users, Goals, Scope, Success Metrics, Constraints	<i>worked / what-we-won't-do</i> decomposition used across major PM frameworks (Cagan's INVEST, Amazon's PR/FAQ, Atlassian PRD template). Missing any one leaves a known failure mode.	
5	Clarifying questions should be batched, not drip-fed	Asynchronous back-and-forth across many messages stretches clarification over days. A single consolidated questionnaire respects the PO's time and produces a complete artifact in one pass.	Medium

4. Analysis

4.1 Pros of the Structured-Intake Approach

- **Unblocks the team in one round-trip.** PO fills the template once; work can start.
- **Creates an audit trail.** The completed template becomes the canonical PRD — no ambiguity about "*what did we agree to?*"
- **Surfaces hidden assumptions early.** Forcing the PO to write success metrics often reveals the feature isn't fully thought through, *before* engineering invests.

4.2 Cons & Risks (with Mitigations)

Risk	Mitigation
PO perceives the template as bureaucratic pushback	Frame as a "5-minute form to unblock your team"; pre-fill any fields you can infer (target release, owning squad).
Template feels heavy for tiny features	Mark fields as Required (*) vs. Optional, so a 1-day bug fix doesn't need a market-sizing section.

Risk	Mitigation
Template substitutes for conversation	Schedule a 15-minute review call after the PO submits, to probe weak answers.

4.3 Alternatives Considered

- **Reject the PRD outright and ask PO to resubmit.** *Excluded* — provides no guidance on what "good" looks like; likely to produce another low-quality draft.
 - **Have engineering draft the PRD and ask PO to approve.** *Excluded* — inverts ownership; engineering shouldn't be inventing product requirements.
 - **Schedule a discovery workshop instead of a written template.** *Reasonable for large features, but overkill if the feature is small. Recommend the template first; escalate to a workshop if answers reveal complexity.*
-

5. Deliverable A — Clarifying Questions for the Product Owner

Send these grouped by section. * = **blocking** (cannot start work without).

5.1 Problem & Context

1. * What user or business problem does this feature solve? (*1-3 sentences*)
2. * What evidence do we have this problem is real? (*support tickets, user research, revenue data, etc.*)
3. What happens if we do *nothing*?
4. Is this net-new functionality, an enhancement, or a fix?

5.2 Users & Roles

1. * Who is the primary user? (*specific persona or role, not "users"*)
2. Are there secondary users or affected roles (*e.g., admins, support staff*)?
3. * What permissions / access level does each role need for this feature?
4. Approximately how many users are in each role today?

5.3 Goals & Scope

1. * What are the top 1–3 outcomes this feature must achieve?
2. * List the **must-have** capabilities for v1.
3. List **nice-to-have** capabilities explicitly deferred to v2+.
4. * What is explicitly **out of scope**? (*this prevents scope creep*)
5. Are there dependencies on other teams, systems, or third-party APIs?

5.4 Success Criteria & Metrics

1. * How will we know this feature succeeded 30 / 60 / 90 days post-launch? (*quantitative if possible*)
2. What is the target baseline vs. goal? (*e.g., "task completion 40% → 70%"*)
3. What is the *failure* threshold that would trigger a rollback or rethink?

5.5 Constraints

1. * Target launch date or release window? Is the date hard or soft?
2. Budget, headcount, or compute constraints?
3. * Compliance, security, accessibility (WCAG), or data-residency requirements?
4. Known technical constraints (*must run on legacy stack X, must integrate with system Y*)?

5.6 Risks & Open Questions

1. What's the riskiest assumption in this feature?
2. What questions do *you* (the PO) still have unanswered?

6. Deliverable B — Fill-In PRD Template